



TED UNIVERSITY

CMPE 492 / SENG 492 Senior Project II

Center of Gravity Detection, Information, and Balancing System in Aircraft

Low Level Design Report

22.03.2026

Team Members:

Selin Göç, Computer Engineering

Elif Sude Memiş, Software Engineering

Sude İpekci, Computer Engineering

Yasin Anber, Computer Engineering

Supervisor:

Prof. Dr. Hakkı Gökhan İLK

Jury Members:

Dr. Ali BERKOL

Dr. Ayşe Yasemin SEYDİM

Table of Contents

1. Introduction	4
1.1 Purpose of the Document	4
1.2 Scope of the System	4
1.3 Design Rationale	4
1.4 Definitions, Acronyms and Abbreviations	4
2. Object Design Trade-offs	5
3. Interface Documentation Guidelines	6
4. Engineering Standards	7
4.1 Unified Modeling Language (UML)	7
4.2 IEEE Documentation Format	7
4.3 Arduino Coding Conventions	7
4.4 Modular Design	8
4.5 Layered Architecture	8
4.6 Hardware–Software Separation	8
5. Packages (Subsystem Decomposition)	8
5.1 Overview of Subsystem Architecture	8
5.2 Subsystem Packages and Their Responsibilities	8
5.3 Package Descriptions	9
5.4 Package Interaction Diagram	10
6. Class Interfaces (Software Components)	11
6.1 SensorModule :Sensor Management and Filtering	12
6.2 CGCalculationModule (Center of Gravity Calculation)	13
6.3 CGVisualizerApp (Visualization and Analysis Interface)	14
6.4 ControlModule (Balancing Decision and Control Logic)	14
6.5 MotorControlModule (Actuation and Hardware Interface)	15
7. Component Design	16
7.1 Failure Handling and Edge Cases	17
8. CG Calculation Algorithm	18
9. Control Algorithm	19
9.1 Real-Time Operation	19
10. Performance Evaluation and Future Work	19
10.1 Design Validation	19

10.2 Future Improvements.....	20
11. Glossary	20
12. References	21

1. Introduction

1.1 Purpose of the Document

The purpose of this report is to provide a detailed low-level design for the Aircraft Center of Gravity Detection and Balancing System, a system developed under the context of the CMPE 492 Senior Design Project. This low-level design is based on a high-level architecture presented in CMPE 491 and further refines the system to specific hardware and software components required for implementation. This report provides an insight into the internal composition of the system, including its subsystems, interfaces, control, and hardware-software integration. In addition, it discusses the rationale and standards applied in the design for the effective and efficient performance of the system. This low-level design is intended to offer a detailed and comprehensive blueprint for the effective development, implementation, and testing of the Aircraft Center of Gravity Detection and Automatic Balancing Mechanism.

1.2 Scope of the System

The proposed system would be able to accurately measure the weight distribution of the aircraft and determine the corresponding center of gravity (CG). It would also be able to automatically adjust the center of gravity of the aircraft using an active balancing system. It would achieve this through a series of force measurements taken from various sensors placed in different areas of the aircraft. It would use these force measurements to accurately determine the precise location of the center of gravity of the aircraft and determine whether it falls within the acceptable limits. If a problem arises, the proposed system would use a controlled balancing system to adjust the position of a counterweight in the aircraft in order to achieve the optimal center of gravity. It would achieve this through the integration of various domains of engineering, including sensing technology used in accurate data acquisition systems.

1.3 Design Rationale

The aircraft's center of gravity is continuously monitored and corrected by the system's closed-loop control architecture. This method offers greater precision and resilience against changes in weight distribution as compared to open-loop systems.

The use of three load cells in a multi-sensor configuration is required to measure the weight distribution of various support points. The use of an embedded microcontroller-based system is required to enable real-time processing and control in an efficient and simple manner.

Accurate CG displacement correction is made possible by the balancing mechanism's two degrees of freedom in the X and Y axes.

1.4 Definitions, Acronyms and Abbreviations

CG (Center of Gravity): The point at which the total weight of the aircraft is considered to act, and around which the aircraft is balanced.

MCU (Microcontroller Unit): A small embedded computing device that processes sensor data, executes control algorithms, and manages system operations.

HX711: A high-precision load cell amplifier and analog-to-digital converter module used to amplify and digitize the very small voltage signals produced by load cell sensors.

PID (Proportional–Integral–Derivative) Controller: A feedback control algorithm that continuously adjusts the system output by minimizing the difference between the desired and measured values.

ADC (Analog-to-Digital Converter): An electronic component that converts analog signals from sensors into digital values that can be processed by a microcontroller or computer.

2. Object Design Trade-offs

During the low-level design phase, various alternatives were considered for the hardware components of the Aircraft Center of Gravity Detection and Balancing System. This was done in an attempt to select the most appropriate solution for the system. The selection of the most appropriate solution was based on various factors that included accuracy of measurement, system simplicity, cost, and ease of integration. Finally, the most appropriate alternatives were selected for the system.

Sensor Selection: To determine the aircraft weight distribution, two different sensing techniques were considered: load cells and force sensors. These two techniques were considered based on their accuracy in measuring weight.

Option	Advantages	Disadvantages
Load Cells	High precision and reliability for weight measurement	Requires signal amplification
Force Sensors	Easier integration and simpler interface	Lower measurement accuracy

Although force sensors offer simpler integration, load cells provide significantly higher precision in detecting weight distribution. Since accurate measurement is critical for calculating the aircraft's center of gravity, load cells were selected as the primary sensing component. The small voltage signals produced by the load cells are amplified using the HX711 amplifier module before being processed by the microcontroller.

Microcontroller Selection: The microcontroller is the main processing unit in the system. It is responsible for reading the data from the sensors, carrying out the CG calculation, and controlling the balancing mechanism. The available platforms for this system were the Raspberry Pi and Arduino Uno.

Option	Reason
Raspberry Pi	Offers higher computational power but introduces unnecessary complexity for this application
Arduino Uno	Provides sufficient processing capability, simple integration with sensors, and reliable real-time control

However, the Raspberry Pi has more processing power. The system does not need a powerful processor. The Arduino Uno board has been chosen for its processing power for sensor data processing in real time while still being low-cost, easy to program, and compatible with load cells and servo motors.

3. Interface Documentation Guidelines

This section defines how the subsystems within the Automatic Center of Gravity (CG) Detection, Monitoring, and Balancing System communicate with each other. The system consists of multiple layers including sensors, signal conditioning modules, an embedded control unit, actuators, and a computer interface. Each interface is designed to ensure reliable, real-time data exchange and proper system operation.

Sensor Interface (HX711 – Arduino)

Signal	Description
DT	Data output from HX711 to Arduino (24-bit digital measurement)
SCK	Clock signal generated by Arduino for HX711

Communication between the HX711 module and the Arduino is performed via a two-wire synchronous serial interface using the DT (Data) and SCK (Clock) lines. Although this communication method operates similarly to the Serial Peripheral Interface (SPI) protocol, it does not use the standard SPI lines such as MOSI, MISO, or CS. Instead, the Arduino generates clock pulses on the SCK line, and the HX711 outputs one bit of data per pulse through the DT line until a full 24-bit measurement is transmitted. This mechanism enables high-resolution acquisition of force measurements obtained from the load cells.

Servo Interface

Signal	Description
PWM	Control signal for servo motor position

Servo motors are controlled using Pulse Width Modulation (PWM) signals generated by the Arduino. By adjusting the pulse width of the PWM signal, the angular position of each servo motor can be precisely controlled. This allows the balancing mass to be moved accurately along the X and Y axes according to the calculated CG deviation.

Serial Communication Interface (Arduino – Computer)

Interface	Purpose
UART	Real-time data transmission and monitoring on a computer

The Arduino Uno transmits processed measurement data — including total weight, load distribution, and CG coordinates — to a computer via UART-based asynchronous serial communication. This communication is implemented through the USB interface, which appears as a virtual serial port on the computer. Real-time monitoring can be performed using serial terminal software or a custom graphical interface application.

HX711–Arduino Communication Overview

The HX711 module amplifies the low-level analog signals produced by the load cells and converts them into high-resolution digital data. The Arduino periodically reads these measurements using the synchronous DT-SCK interface, applies calibration and filtering algorithms, and forwards the processed values to the CG calculation module. This communication channel forms the primary data path between the sensing layer and the embedded control system, serving as the starting point for the entire balancing process.

Low-Level Design Considerations

The low-level design of the system focuses on the acquisition of physical measurements from sensors, their processing through hardware interfaces, and the application of control actions via actuators. Commercial off-the-shelf components — including load cells, the HX711 module, Arduino Uno microcontroller, and servo motors — were selected to provide a reliable and cost-effective implementation platform with a high level of abstraction from raw hardware.

During the analysis phase, the system objectives were defined. In the system design phase, the architecture, subsystem decomposition, and global control flow were established. At this stage, corresponding to object design in computer engineering, solution components such as sensor acquisition modules, data processing routines, decision logic, and actuator control mechanisms were specified in detail, and the interfaces between subsystems were clearly defined.

This structured approach ensures compatibility between hardware and software components while enabling modularity, scalability, and maintainability of the embedded system.

4. Engineering Standards

4.1 Unified Modeling Language (UML)

The Unified Modeling Language (UML) is used as the modeling language to represent the system structure and behavior. UML diagrams are used to represent the system components, their relationship, and their interaction. The UML diagrams are used to effectively communicate complex system design. In the context of the current project, UML is used to represent the system design of the Aircraft Center of Gravity Detection and Balancing System. The relationship between the sensing module, control module, and actuation subsystem is also represented using UML diagrams. UML is used to effectively translate ideas into system design. UML is also used to ensure that all team members have a common understanding of the system design during the implementation phase.

4.2 IEEE Documentation Format

The format used for the project documentation is based on the IEEE format for a technical report. This format is commonly used for engineering or programming projects. The IEEE format helps ensure that the documentation is presented in a clear, logical, and professional manner. This includes sections for the introduction, design decisions, subsystem description, etc. This format helps ensure that the readability of the documentation is high, while at the same time ensuring that it meets the requirements for a technical report.

4.3 Arduino Coding Conventions

In terms of the implementation of embedded software, it follows the coding conventions and best practices for Arduino programming. This includes a well-structured program, a modular function, a descriptive

variable and function name, and inline documentation, among others. This ensures that the embedded software is more readable, maintainable, and reliable, making it easy for anyone who may be needed in the future for improvement or modification of the software.

4.4 Modular Design

The system design is carried out in a modular fashion, where the entire system functionality is broken down into smaller, independent modules. Each module in the system is dedicated to a certain function, for example, acquiring sensor information, CG calculation, control, and motor control, among others. This makes system development more manageable, as well as eases debugging, as the system is broken down into well-defined modules. In addition, a modular system design makes it possible for a module in a system to be easily changed or upgraded without affecting the entire system.

4.5 Layered Architecture

The layered architecture is used to organize the system into logical functional levels. The sensing layer is responsible for raw data acquisition using load cells. The processing layer is responsible for signal conditioning and CG calculation. The control layer is responsible for the control of the balancing mechanism using a servo motor. This layered architecture provides a high level of clarity to the system.

4.6 Hardware–Software Separation

Another aspect of system design is the separation of hardware and software components. In this system, hardware components like load cells, HX711 amplifiers, and servo motors are responsible for interacting with the system environment, while software components running on the microcontroller are responsible for processing sensor information and making control decisions. The separation of hardware and software components makes a system more flexible and scalable, as hardware components can be easily upgraded without affecting software components.

5. Packages (Subsystem Decomposition)

5.1 Overview of Subsystem Architecture

To further enhance system organization, maintainability, and scalability, the Aircraft Center of Gravity Detection and Balancing System is divided into several functional subsystems, which are called packages. Each package is responsible for a certain part of the system's functionality and interacts with other packages via well-defined interfaces.

This modular decomposition of a subsystem is useful in developing, testing, and maintaining a system efficiently. This is because, by dividing a complex system into several functional units, the complexity of the system is reduced, as well as the complexity of interactions between different components of the system.

5.2 Subsystem Packages and Their Responsibilities

Package	Function
Sensor Package	Collects weight data from load cells placed under the aircraft landing points

Package	Function
Signal Processing Package	Amplifies and converts analog sensor signals into digital data
Control Package	Calculates the center of gravity and determines balancing actions
Actuation Package	Moves the counterweight using servo motors
Monitoring Package	Sends system data to the user interface for monitoring

5.3 Package Descriptions

Sensor Package:

The job of the Sensor Package is to obtain the measurements of the weight from the aircraft structure. The load cells, which are placed under the landing points of the aircraft, measure the force exerted on the aircraft, which is converted to electrical signals. The signals represent the allocation of the aircraft's weight on the points supporting the aircraft.

This system is the main source of information for the whole system, as the measurements of the weight are necessary for calculating the center of gravity of the aircraft.

Main components

- Load cells
- Wheatstone bridge structure
- Mechanical mounting structure

Signal Processing Package:

The Signal Processing Package converts the small analog signals from the load cells and makes them digital for the microcontroller to process. The load cells produce very small analog signals in the form of voltage. The signals need to be amplified before being converted to digital signals.

The HX711 module is used for amplification and analog-to-digital conversion.

Main components

- HX711 amplifier modules
- Analog-to-digital conversion circuit

Control Package:

The Control Package serves as the central processing unit of the system. The digital weight measurements from the signal processing subsystem are used to determine the center of gravity of the aircraft using mathematical formulas according to the moment principle.

The control module will check if the center of gravity is within the range of operation. When there is a detected imbalance, the control module sends control signals to activate the balancing mechanism.

Main responsibilities

- CG calculation
- error detection
- control decision generation

Hardware component

- Arduino Uno microcontroller

Actuation Package

The role of the Actuation Package is to adjust the center of gravity of the aircraft model. This is done by moving a counterweight along specific axes to achieve optimal position.

Servo motors are used to move the lead screws to achieve linear motion.

Main components

- Servo motors
- Lead screw mechanism
- Counterweight platform

Monitoring Package:

The Monitoring Package is used for system feedback, where users can view the status of the aircraft balancing system through readings of sensor data, CG coordinates, and system operation status.

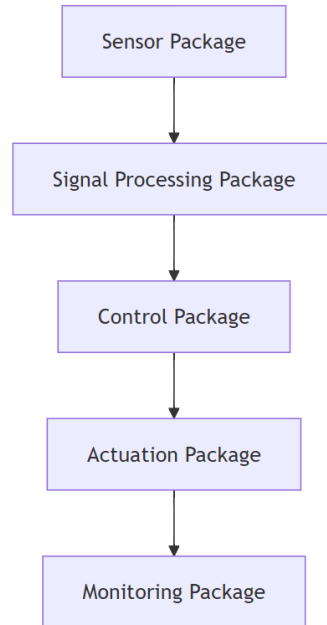
The communication between the microcontroller and monitoring interface is done through serial communication.

Functions

- transmit sensor data
- display CG coordinates
- monitor system performance

5.4 Package Interaction Diagram

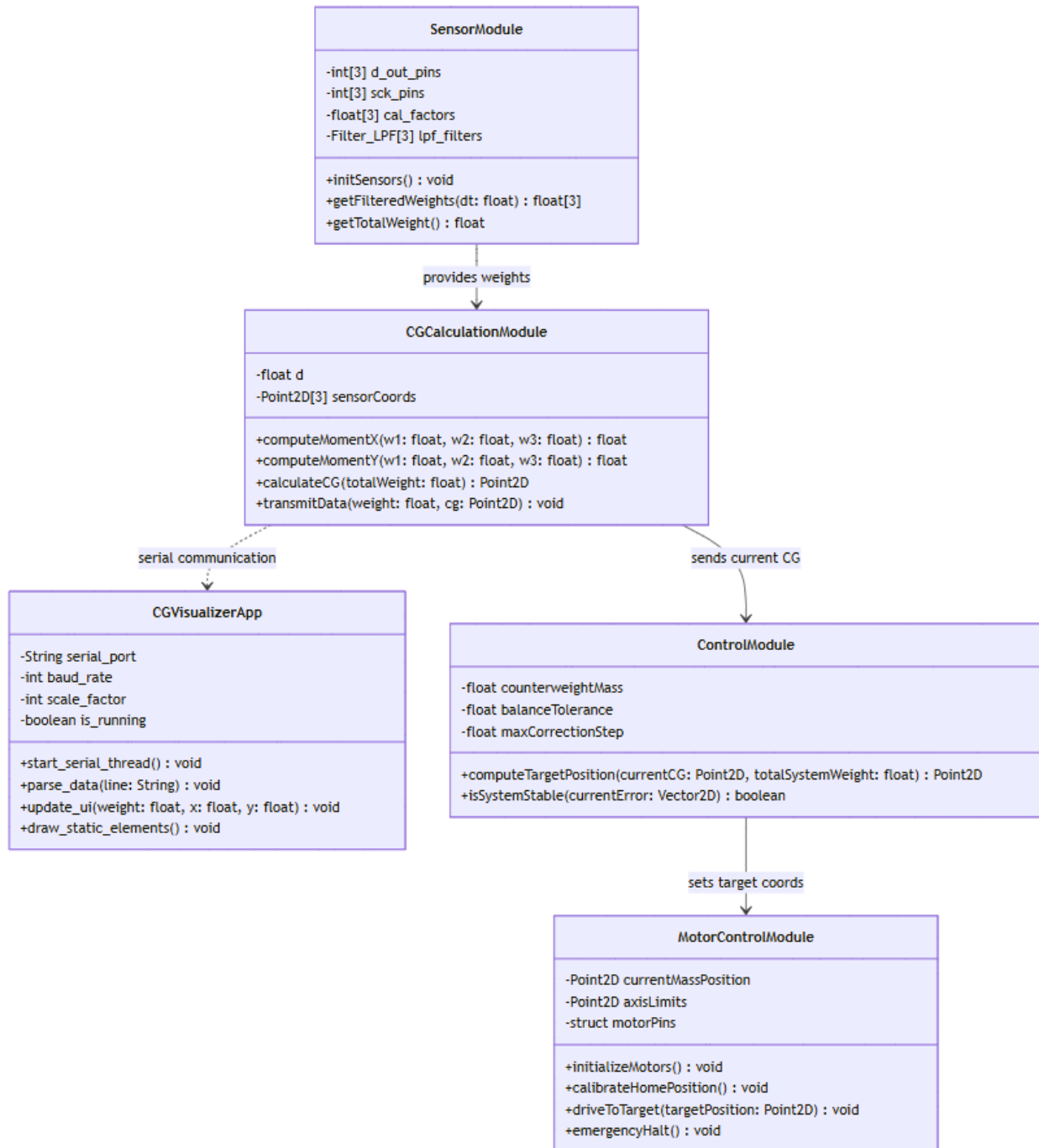
The interaction between the subsystems follows a sequential flow of data and control.



In this architecture, the sensor subsystem continuously collects weight data, which is processed and analyzed by the control subsystem. If the calculated center of gravity deviates from the desired position, the actuation subsystem adjusts the counterweight accordingly. Meanwhile, the monitoring subsystem provides real-time feedback about system operation.

6. Class Interfaces (Software Components)

The software architecture of the “Center of Gravity Detection and Balancing System for Aircraft” project is modularized in accordance with the Separation of Concerns principle, encompassing both the embedded system (C/C++) and the computer interface (Python). The fundamental software components and class interfaces of the system are defined below.



6.1 SensorModule :Sensor Management and Filtering

This class is responsible for establishing hardware-level communication with the 3 HX711 load cell amplifiers in the system. It reads electrical data from the sensors, calibrates it, and applies a Low Pass Filter (LPF) to increase system stability.

Responsibilities:

- Read synchronized data from the HX711 modules (Right, Left, and Front Legs).

- Perform automatic tare operations upon system initialization.
- Apply a Low Pass Filter (LPF) with a cutoff frequency of 0.05f to the raw readings to eliminate sudden spikes (noise) caused by mechanical vibrations.
- Implement deadzone control, treating vibrations below ± 1.0 grams as zero (0.0) to prevent micro-fluctuations.

Attributes:

- **-d_out_pins: int[3]** (Sensor data pins: 2, 4, 6)
- **-sck_pins: int[3]** (Sensor clock pins: 3, 5, 7)
- **-cal_factors: float[3]** (Calibration multipliers: 393.5, -409.8, 383.7)
- **-lpf_filters: Filter::LPF<float>[3]** (Filter objects)

Methods:

- **+ initSensors(): void** (Configures pins, sets calibration values, and tares the scales)
- **+ getFilteredWeights(dt: float): float[3]** (Returns 3 filtered weight values based on the elapsed time)
- **+ getTotalWeight(): float** (Calculates the sum of the filtered weights)

6.2 CGCalculationModule (Center of Gravity Calculation)

Utilizing the filtered weight data received from the sensor module and the known geometric (X, Y) coordinates of the sensors on the platform, this module calculates the instantaneous Center of Gravity (CG) using moment equilibrium equations.

Responsibilities:

- Establish the static coordinate matrix based on the sensors' distance from the center and placement angles
- Bypass the calculation when the total weight is near zero to prevent Divide by Zero errors (Threshold: > 10.0 g).
- Execute the moment formula ($M = F \cdot d$) to generate instantaneous X_{cg} and Y_{cg} values.
- Format the calculated data into a structure suitable for serial communication (Serial.print).

Attributes:

- **- d: float** (Fixed (It will be determined based on the layout of the helicopter prototype's landing gear) distance from the center to the sensors)
- **- sensorCoords: Point2D[3]** (X, Y coordinates of the Front, Right, and Left legs)

Methods:

- **+ computeMomentX(w1: float, w2: float, w3: float): float**
- **+ computeMomentY(w1: float, w2: float, w3: float): float**
- **+ calculateCG(totalWeight: float): Point2D** (Returns the computed X_{cg} and Y_{cg} points)

- + **transmitData(weight: float, cg: Point2D): void** (Writes the data to the serial port in the format: weights:w g , X_CG: m , Y_CG:n)

6.3 CGVisualizerApp (Visualization and Analysis Interface)

This is a Python-based desktop application developed using the Tkinter library. It asynchronously listens to the serial port data coming from the embedded system in real-time, parses it, and renders it onto a 2D graphical display understandable by the user.

Responsibilities:

- Initiate an asynchronous data listening loop (Thread) over the specified COM port (e.g., COM19).
- Parse the incoming String formatted packets to extract Weight ;X and Y variables.
- Draw the calculated instantaneous CG point as a moving dot (red ball) on the graphical plane and visually scale the system coordinates (scale_factor = 25).
- Provides dynamic visual feedback regarding system stability by monitoring the CG's spatial displacement from the origin. The graphical marker turns red if the positional error exceeds the ± 2.0 threshold, and green to signify a balanced state

Attributes:

- - **serial_port: String** (The communication port to listen to)
- - **baud_rate: int** (Communication speed: 9600 bps)
- - **scale_factor: int** (Visual zoom multiplier for the Canvas)
- - **is_running: boolean** (Thread control flag)

Methods:

- + **start_serial_thread(): void** (Assigns the serial port reading process to a background daemon thread)
- + **parse_data(line: String): void** (Parses the incoming line of data)
- + **update_ui(weight: float, x: float, y: float): void** (Updates the Tkinter interface components in a Thread-Safe manner)
- + **draw_static_elements(): void** (Draws the coordinate axes and the reference Datum point)

6.4 ControlModule (Balancing Decision and Control Logic)

This module functions as the central decision-making unit of the automated balancing system. Instead of complex closed-loop algorithms like PID, it relies on direct static moment equations and threshold-based logic. It evaluates the instantaneous Center of Gravity (CG) displacement and calculates the exact counter-movement required by the balancing mass to restore equilibrium.

Responsibilities:

- Continuously evaluate the spatial error (Delta_X and Delta_Y) between the current CG and the target equilibrium point (0, 0).
- Calculate the target physical coordinates for the moving counterweight using direct mass-moment equations (e.g., $m_{\text{system}} \cdot x_{\text{cg}} + m_{\text{counterweight}} \cdot x_{\text{target}} = 0$).

- Implement a deadband (tolerance) logic so that the balancing mechanism does not continuously hunt for micro-fluctuations, thereby saving power and reducing mechanical wear.
- Transmit the computed specific target coordinates to the MotorControlModule.

Attributes:

- - **counterweightMass: float** (The known physical mass of the moving balancing unit in grams)
- - **balanceTolerance: float** (The acceptable radius of deviation from the center where the system is considered "balanced")
- - **maxCorrectionStep: float** (Optional limit for the maximum allowed movement per iteration to ensure mechanical safety)

Methods:

- + **computeTargetPosition(currentCG: Point2D, totalSystemWeight: float): Point2D** (Calculates the exact required mathematical position for the counterweight to neutralize the current CG offset)
- + **isSystemStable(currentError: Vector2D): boolean** (Returns true if the calculated error falls within the defined balanceTolerance)

6.5 MotorControlModule (Actuation and Hardware Interface)

This class serves as the hardware abstraction layer for the physical actuators driving the X-Y balancing mechanism. It translates the target coordinates generated by the ControlModule into electrical signals (such as Step/Direction pulses or PWM signals) to physically displace the counterweight.

Responsibilities:

- Generate precise control signals to drive the motors along the X and Y axes.
- Track the real-time physical position of the moving mass relative to its starting point (home position).
- Enforce strict software and hardware constraints (using limit switches/endstops) to prevent the mechanism from crashing into the helicopter's chassis.
- Perform an initial homing sequence (calibration) upon startup to establish an accurate (0, 0) reference frame for the counterweight.

Attributes:

- - **currentMassPosition: Point2D** (The tracked real-world position of the counterweight)
- - **axisLimits: Point2D** (The maximum allowable travel distance in millimeters or steps for both X and Y axes)
- - **motorPins: struct** (Configuration of microcontroller pins mapped to motor drivers)

Methods:

- + **initializeMotors(): void** (Configures output pins and prepares motor drivers)

- + **calibrateHomePosition(): void** (Drives the mechanism slowly to the limit switches to zero the coordinate system)
- + **driveToTarget(targetPosition: Point2D): void** (Moves the balancing mass precisely to the newly calculated target coordinate provided by the Control Module)
- + **emergencyHalt(): void** (Immediately cuts off motor movement in case of a critical error or manual override)

7. Component Design

Component 1 – Load Cell Sensing Subsystem

The Load Cell Sensing Subsystem is in charge of detecting the aircraft's weight distribution. The load cells are located at the support points of the aircraft's structure and measure the force applied to these points. The load cells use strain gauge technology to perform their sensing. The strain gauges measure the deformation caused by the aircraft's weight and the changes in electric resistance. The Wheatstone bridge configuration measures the changes in electric resistance and sends a voltage proportional to the force. The voltage represents the weights at the different points where the load cells are located and is mainly used for calculating the aircraft's center of gravity.

Function:

- measure aircraft weight distribution.

Component 2 – Signal Conditioning Subsystem

The Signal Conditioning Subsystem receives the low analog voltage signals from the load cells and conditions these signals for further processing in a digital format. The signal from a load cell is in microvolts, and hence a signal amplifier is necessary for further processing by a microcontroller. This is achieved in the HX711 module, which amplifies the load cell signals and converts these analog signals into a digital format using a 24-bit analog-to-digital converter for further processing in a microcontroller using synchronous serial communication for accurate weight measurement in the control loop.

HX711 amplifier:

- amplifies microvolt signals
- converts analog signals to digital values

Data is transmitted to Arduino via synchronous communication.

Component 3 – Control Unit

The Control Unit is the processing core of this system, where an Arduino Uno microcontroller is utilized based on ATmega328P architecture. This unit processes digital weight readings provided by the signal conditioning system in real-time. The primary functions of this control unit are to process sensor readings, compute the aircraft's center of gravity through specific formulas, and validate if it is within acceptable limits. In case of an imbalance, this control unit will send control signals to trigger the balancing mechanism. This control unit is therefore the decision-making heart of this system.

Microcontroller: Arduino Uno (ATmega328P)

Responsibilities:

- read sensor data
- compute CG coordinates
- control servo motors

Component 4 – Balancing Mechanism

The Balancing Mechanism is responsible for making physical changes to the aircraft's center of gravity whenever an unbalance is sensed. The Balancing Mechanism is made up of servo motors, lead screws, and a counterweight platform. The servo motors are used to operate the lead screws, which are capable of converting rotational motion to linear motion. This is done to ensure precise movement of the counterweight platform along the X-axis and Y-axis. The movement of the platform along the X-axis is for horizontal movement, while movement along the Y-axis is for vertical movement. The movement of the platform is to adjust the center of gravity towards a balanced position.

Mechanical system including:

- servo motors
- lead screw mechanism
- movable counterweight

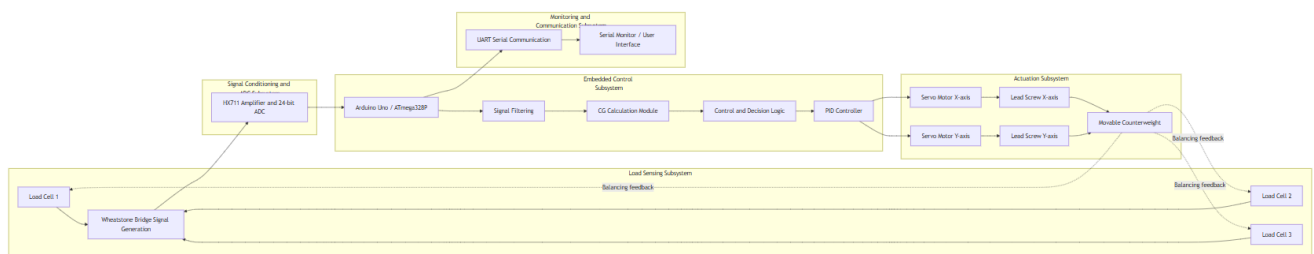


Figure 1.

Figure 1 shows the detailed low-level architecture of the Aircraft Center of Gravity Detection and Balancing System. The load sensing subsystem detects the weight distribution using three load cells, which are then converted into electrical signals using Wheatstone bridge configurations. These electrical signals are then amplified and converted into digital signals by the HX711 module before being processed by the Arduino-based embedded control subsystem. The signals are then filtered, the center of gravity is calculated, the deviation from the balanced point is determined, and the control signals are then used by the actuation subsystem to move the counterweight along the X and Y axes using servo motors. At the same time, the monitoring subsystem sends the measurement signals in real-time to the user interface using serial communication.

7.1 Failure Handling and Edge Cases

The system has procedures to deal with possible failure situations. Filtering methods like dead zones and low-pass filtering minimize sensor noise and unstable readings. In order to avoid inaccurate CG calculations, the algorithm ignores outliers if a sensor delivers incorrect or inconsistent data. The counterweight is kept from going beyond safe movement limits by enforcing mechanical constraints. The system stops motion in the event of an actuator failure to prevent damage. These safety measures guarantee safe operation and increase reliability of the system.

8. CG Calculation Algorithm

CG is calculated using the moment principle.

Total weight

- The total weight of the aircraft model is calculated by summing the forces measured by all load cells. Each load cell measures the vertical force applied at its specific mounting location. Since the aircraft structure is supported by three load cells in the system, the total weight W is obtained by adding the individual force readings from each sensor.

$$W = F_1 + F_2 + F_3$$

- In this equation, F_1 , F_2 and F_3 represent the forces measured by the three load cells. These forces correspond to the weight distribution at different support points of the aircraft. Calculating the total weight is an essential step in the CG calculation process, as it is used as the denominator in the center of gravity formulas. Accurate weight measurement ensures that the CG position can be computed reliably.

CG position

- The position of the center of gravity is determined based on the moment principle. The moment principle is based on the idea that the position of the CG can be determined by dividing the total moment by the total weight. The load cells contribute to the calculation of the position of the CG based on the weight that is determined by the magnitude of the forces that the load cells measure. The position of the load cells is based on the coordinate system of the aircraft model.

$$CG_x = \frac{\sum(F_i \cdot x_i)}{W}$$

$$CG_y = \frac{\sum(F_i \cdot y_i)}{W}$$

- In these equations, F_i represents the force measured by the i^{th} load cell, while x_i and y_i represent the coordinates of that sensor in the aircraft's reference frame. The term $F_i \cdot x_i$ or $F_i \cdot y_i$ corresponds to the moment generated by the measured force around the respective axis. By summing the moments of all sensors and dividing by the total weight W , the precise coordinates of the center of gravity are obtained. These calculated coordinates allow the control system to determine whether the aircraft is balanced or whether the counterweight mechanism must be activated to restore the desired CG position.

9. Control Algorithm

The Aircraft Center of Gravity Detection and Balancing System utilizes a closed-loop control algorithm to ensure continuous monitoring of the position of the center of gravity. A closed-loop system is one where the system's output is continuously measured or monitored against the desired value or reference. If any difference is observed, appropriate measures are taken to minimize the difference or error to bring the system to the desired condition or state. This enables the system to ensure balance even when the load is not evenly distributed.

The control process begins with the measurement of data by the load cells, whereby the load cells sense the forces representing the aircraft's weight distribution. The data is then amplified and digitized by the signal conditioning subsystem before it is fed to the microcontroller.

The center of gravity (CG) is then calculated based on the sensed forces and the coordinates of the load cells. The position of the CG is determined on the X and Y axes based on the moment equations presented above in CG Calculation Algorithm section.

After the calculation of the CG position, the system conducts an error-detecting phase. This phase compares the CG coordinates obtained from the measurement to the desired reference CG position. If the error is found to exceed the set tolerance limit, the aircraft is considered to be unbalanced.

For the correction of the unbalance condition, a PID control algorithm is used. This algorithm determines the control signals according to the error value. This ensures smooth control without the occurrence of oscillatory movements.

The control signals are then sent to the servo motors. The motors are responsible for moving the counterweight along the X and Y axes using the lead screw. This movement of the mass distribution of the system helps to move the CG to the desired location.

The entire process repeats itself in a cycle. The process is a feedback loop that continuously checks and adjusts the CG position. The process repeats itself until the calculated CG position is within an acceptable range.

9.1 Real-Time Operation

The system operates in real-time by executing the control loop periodically. The sensor continuously calculates CG while sampling data on a regular basis.

The control loop is built to be able to identify any deviation in the CG position and make the necessary adjustments rapidly. In doing so, the system will remain stable.

10. Performance Evaluation and Future Work

10.1 Design Validation

The proposed system design will be validated through a combination of experimental testing and performance analysis. Different weight distributions will be applied to the aircraft model in order to evaluate the accuracy of the center of gravity (CG) calculation. The measured CG values will be compared with expected theoretical values to verify correctness.

In addition, the performance of the balancing mechanism will be tested by introducing controlled imbalances and observing the system's response. Key evaluation criteria include the accuracy of CG correction, response time of the system, and stability of the control algorithm. The system is considered successful if it is able to bring the CG within the acceptable tolerance range in a consistent and stable manner.

10.2 Future Improvements

Several improvements can be made to enhance the performance and capabilities of the system. Higher precision sensors can be used to increase measurement accuracy, and more advanced microcontrollers can be integrated to support faster processing and more complex control algorithms.

Wireless communication modules may be added to enable remote monitoring and control of the system. In addition, more advanced control strategies such as adaptive control or machine learning-based approaches can be explored to improve balancing performance under dynamic conditions.

Finally, the current system can be extended from a two-dimensional balancing mechanism to a three-dimensional system, allowing more realistic and comprehensive aircraft CG management.

11. Glossary

Center of Gravity (CG): The point where the total weight of the aircraft is balanced.

Load Cell: A sensor that measures force or weight and converts it into an electrical signal.

Embedded System: A dedicated computing system designed to perform specific control or monitoring tasks.

Microcontroller Unit (MCU): A small processing unit that reads sensor data and executes control algorithms.

HX711 Amplifier: A module that amplifies load cell signals and converts them into digital data.

PID Controller: A control algorithm used to adjust system behavior based on error between desired and measured values.

Analog-to-Digital Converter (ADC): A device that converts analog signals into digital values for processing.

Servo Motor: A motor used to control precise movement in the balancing mechanism.

Lead Screw Mechanism: A mechanical system that converts rotational motion into linear movement.

Signal Conditioning: The process of amplifying and preparing sensor signals for digital processing.

12. References

- [1] Roskam, J. Airplane Flight Dynamics and Automatic Flight Controls, Part I. DARcorporation, 1995.
- [2] Federal Aviation Administration (FAA). *Weight and Balance Handbook*, FAA-H-8083-1B, 2016.
- [3] Cook, M. V. *Flight Dynamics Principles*, 3rd Edition. Butterworth-Heinemann, 2012.
- [4] Li, Y., Wang, J., & Zhang, H. “Aircraft Weight and Balance Measurement System Based on Sensors.” *Journal of Aircraft*, Vol. 54, No. 3, 2017.
- [5] Bolton, W. *Mechatronics: Electronic Control Systems in Mechanical and Electrical Engineering*, 6th Edition. Pearson Education, 2015.